

Experiment 7: Quick Sort

Aim

To implement the **Quick Sort** algorithm using Python to sort a list of elements in ascending order.

Algorithm

Step 1: Start.

Step 2: Read the number of elements and the array.

Step 3: Select the last element as the pivot.

Step 4: Place all elements smaller than the pivot on its left and larger elements on its right.

Step 5: Recursively apply Quick Sort to the left and right subarrays.

Step 6: Display the sorted array.

Step 7: Stop.

Source Code (Python)

```
main.py
1  # Quick Sort
2
3  def quick_sort(arr):
4      if len(arr) <= 1:
5          return arr
6
7      pivot = arr[-1]
8      left = []
9      right = []
10
11     for i in arr[:-1]:
12         if i <= pivot:
13             left.append(i)
14         else:
15             right.append(i)
16
17     return quick_sort(left) + [pivot] + quick_sort(right)
18
19 n = int(input("Enter number of elements: "))
20
21 arr = []
22
23 print("Enter the elements:")
24 for i in range(n):
25     arr.append(int(input()))
26
27 sorted_arr = quick_sort(arr)
28
29 print("Sorted array:", sorted_arr)
```

Output

```
Output
Enter number of elements: 6
Enter the elements:
50
20
80
10
40
30
Sorted array: [10, 20, 30, 40, 50, 80]

=== Code Execution Successful ===
```

Time Complexity

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n^2)$

Space Complexity

- **$O(\log n)$** (recursive calls)

Result

The **Quick Sort** algorithm was implemented successfully in Python. The program sorted the given array in ascending order using the **Divide and Conquer** technique, and the output was obtained correctly.